



MEDCORE

HOSPITAL MANAGEMENT SYSTEM

Full System & Build Documentation

Modules: Patient Records · Appointments · Billing · Pharmacy · Laboratory · Staff · Reports

Frontend: React (Vite) | Backend: FastAPI (Uvicorn) | Database: PostgreSQL
Auth: JWT (Argon2 password hashing) | Session storage: browser session only
Email: Gmail SMTP, sent from the hospital's own mailbox — no third-party email provider

Prepared for: MEDCORE HMS Development Team

Document Version 4.0 — Final Build

Status: Complete — Single-language build, two-theme system, full role-based access control, hospital-mailbox email delivery

Table of Contents

- 1. Project Overview..... 4
 - 1.1 Introduction..... 4
 - 1.2 Objectives..... 4
 - 1.3 Tech Stack..... 4
 - 1.4 User Roles & Permissions..... 5
 - 1.5 Theming System..... 5
 - 1.6 Local Network Deployment & Email Delivery..... 6
- 2. Frontend Documentation (React)..... 7
 - 2.1 Global Layout Components..... 7
 - 2.2 Authentication Pages..... 8
 - 2.3 Dashboard..... 8
 - 2.4 Patient Records..... 8
 - 2.5 Appointments..... 9
 - 2.6 Billing..... 10
 - 2.6a Refunds..... 10
 - 2.7 Pharmacy..... 11
 - 2.8 Laboratory..... 11
 - 2.9 Staff Management..... 12
 - 2.10 Reports..... 12
 - 2.11 Settings..... 12
 - 2.12 My Profile..... 12
 - 2.13 Notifications..... 13
 - 2.14 Error Page..... 13
- 3. Backend Documentation (FastAPI + Uvicorn)..... 14
 - 3.1 Architecture Overview..... 14
 - 3.2 Database Schema..... 14
 - 3.3 API Endpoints..... 18
- 4. Known Limitations..... 23

Full Feature Index

Every feature and interface element of the completed application, listed here for quick reference. Each item is expanded in full inside the Frontend and Backend sections that follow.

- Theme Toggle — switches between Clinical Light and Night Shift (Dark) themes app-wide, no reload
- Navbar — page title, global search, theme toggle, notification bell with live unread count, quick-add button, role label + profile avatar
- Sidebar navigation — Dashboard, Patients, Appointments, Billing, Pharmacy, Laboratory, Staff Management, Reports, Settings, with the logged-in user's role shown under the brand mark
- Footer — hospital name & copyright, app version, support/contact link
- Authentication — Login, Forgot Password, Reset Password, Set Password (first-time staff activation); session ends when the browser window closes
- Dashboard — key stat cards, today's appointment widget, revenue chart, recent patients table, quick actions
- Patient Records — patient list/search, add/edit patient, patient profile with Overview, Medical History, Appointments, Billing, Lab Results and Prescriptions tabs
- Appointments — hand-built month calendar & list views, book/edit appointment (auto-generates a fixed-fee invoice for revenue tracking), enforced status workflow (only the assigned doctor may confirm or complete a visit), consultation notes, patient email on confirmation
- Billing — invoice list, create/edit invoice with line items, record payments, PDF receipts; every billing endpoint (including read access) is restricted to Admin and Receptionist
- Pharmacy — drug inventory, add/edit drug (adding new drugs is Pharmacist-only), low-stock alerts that generate real notifications, prescriptions list, dispense workflow
- Laboratory — test catalog (add/edit restricted to Lab Technician only), lab order list, a payment gate that links every lab order to an auto-generated invoice, result entry, doctor notification and patient email on completion
- Staff Management — staff list, add/edit staff with roles; first-time passwords are set via an emailed activation link
- Reports — revenue, patients, appointments, pharmacy stock, lab utilization; export to PDF/Excel
- Settings — hospital profile, user preferences, roles & permissions matrix, fixed appointment fees (Admin only), data backup
- Notifications — real, event-driven in-app notifications for low stock, new prescriptions, new lab orders, and completed lab results; live unread badge in the Navbar
- Patient Emails — sent from the hospital's own Gmail mailbox when a doctor confirms an appointment, when a lab result is marked completed, and when an invoice is paid in full (with the PDF receipt attached)
- My Profile — edit own info, change password

1. Project Overview

1.1 Introduction

The MEDCORE Hospital Management System (HMS) is a full-stack web application built to digitize the day-to-day operations of a hospital or clinic. It covers patient record keeping, appointment scheduling, billing, pharmacy stock and dispensing, and laboratory test management, all accessible through a single role-based dashboard.

This document describes the completed, as-built system. An earlier planning draft of this document described a broader scope — multi-language support (i18n) and a three-theme system — both of which were deliberately scoped out of the final build to keep the system focused and defensible. What remains is a single-language (English), two-theme, fully role-gated hospital management system with real business logic behind every workflow described below.

1.2 Objectives

- Replace paper-based patient records with a searchable digital system
- Give front-desk staff a fast way to register patients and book appointments
- Track billing, payments and outstanding balances per patient
- Prevent pharmacy stockouts through low-stock alerts that generate real, targeted notifications
- Give lab technicians a structured, payment-verified workflow from order to result
- Give management real-time reports on revenue, patient flow, and stock
- Enforce strict, server-verified role-based access control on every write action, not just hide buttons in the interface

1.3 Tech Stack

Field	Type	Notes
Frontend	React (Vite)	SPA, component-based UI, calls the backend via REST through a single shared axios client
Backend	FastAPI (Python, Uvicorn server)	REST API, async request handling (SQLAlchemy async + asyncpg), auto-generated OpenAPI docs at /docs
Database	PostgreSQL	Relational schema — see Section 3.2
Auth	JWT (access + refresh tokens)	Issued by POST /api/auth/login, verified per request. Passwords hashed with Argon2 (via pwdlib), not bcrypt
Session storage	Browser sessionStorage	Tokens are cleared the moment the browser window or tab is closed — every role must log in again on next open; no persistent "remember me"
Styling	Tailwind CSS (utility classes generated from a small design-token set)	Consistent design system across every page, all colors read from CSS custom properties
PDF generation	reportlab	Used for invoice receipts and PDF report exports
Spreadsheet export	openpyxl	Used for Excel report exports
Email delivery	Gmail SMTP (Python's built-in smtplib — no third-party email API)	Sent directly from the hospital's own Gmail mailbox using an App Password. Every email attempt is also logged to the backend console as a permanent fallback, and is sent as a background task so a slow or unreachable mail server never blocks the user's action — see Section 1.6

Field	Type	Notes
Migrations	Alembic	Schema changes are applied identically to both hospital databases

1.4 User Roles & Permissions

The system uses strict role-based access control (RBAC), enforced on the backend for every write action — not just hidden or shown in the interface. The table below summarizes each role's access; specific restrictions worth calling out are listed underneath.

Role	Access Summary
Admin	Full read access across the system and most administrative actions (staff, settings, reports). Cannot add or edit lab tests, and cannot add new drugs to inventory — those two actions are deliberately reserved for the specialist roles below, even for Admin. Processing a refund is available to Admin and Receptionist, same as the rest of Billing — but the standalone Refunds audit page (Section 2.6a), which lists and exports every refund ever processed, is Admin-only
Doctor	Views patients and appointments, writes prescriptions, orders lab tests. May confirm and complete an appointment only when they are the specific doctor assigned to that appointment
Nurse	Views patient records and appointments; can cancel or mark an appointment as no-show, and can assist with lab order creation
Receptionist	Registers patients, books appointments; together with Admin, is one of only two roles with any access at all to Billing — including simply viewing an invoice
Pharmacist	The only role permitted to add a new drug to inventory. Also edits stock/drug details (alongside Admin) and dispenses prescriptions
Lab Technician	The only role permitted to add or edit a lab test in the test catalog (price, turnaround time, category). Confirms lab orders once payment clears, submits results, and completes orders

Enforcement note: every restriction above is checked on the backend (a rejected request returns HTTP 403), and mirrored on the frontend so a user is never shown a control they are not permitted to use. The frontend check exists purely for a clean user experience — the backend check is what actually protects the data.

1.5 Theming System

The UI supports two built-in themes, switched at runtime with no page reload. All color values are CSS custom properties, so every component reads color through a variable rather than a hard-coded hex value.

Theme	Type	Notes
light (default)	Clinical Light	Daytime use — pale blue-white surface, deep teal accent
dark	Night Shift	Low-light wards / night rounds — near-black surface, matching teal accent tuned for legibility on a dark background

How it works:

- `index.css` defines the token set (`--hms-bg`, `--hms-surface`, `--hms-surface-2`, `--hms-text`, `--hms-text-dim`, `--hms-accent`, `--hms-accent-on`) twice, once per theme, scoped under `:root[data-theme="..."]`
- `ThemeToggle.jsx` sets the `data-theme` attribute on the `<html>` element; every variable recalculates instantly and every component re-renders with the new palette — no per-component logic needed
- Components consume the tokens as Tailwind utility classes generated from the `@theme` block — e.g. `bg-hms-surface`, `text-hms-accent`, `text-hms-text-dim` — rather than arbitrary `bg-[var(--hms-...)]` values
- Status colors (`--hms-success`, `--hms-warning`, `--hms-danger`, `--hms-info`) are defined once, outside the theme blocks, and stay constant across both themes — a "Cancelled" badge must always read as `danger-red` regardless of which theme is active

Removed from the original plan: a third "Comfort Mode" (Calm) theme was designed and partially built early on, but was removed from the final system along with all of its CSS tokens and constant references, to keep the theme system simple and consistent.

1.6 Local Network Deployment & Email Delivery

The system is designed to run entirely on a hospital's own local server, with every connected computer in the building reaching it over the local network (LAN) rather than the public internet. Email — activation links, password resets, appointment confirmations, lab-result notices, and paid-invoice receipts — is sent directly from the hospital's own Gmail mailbox using Python's built-in `smtplib`, with no third-party email API or paid service involved (an earlier draft of this build used a third-party provider; that dependency has been fully removed).

Two separate things happen when the system sends an email

- Sending the email itself travels over the internet, via Gmail's mail servers — this works from anywhere the server machine has an outbound internet connection, the same as any normal email client
- Any link inside that email (e.g. a "Set your password" link) is just text pointing at a URL. That URL is only reachable by devices on the same local network as the server, and only if it points at an address the whole network can resolve — not the word "localhost", which always means "whichever device opened this link", not the server itself

Correct setup: `CORS_ORIGINS` in `.env` must be set to the server machine's real LAN IP address (e.g. `http://192.168.1.50:5173`), not `localhost` — every emailed link is built from this value. The server machine should be given a fixed IP via a router DHCP reservation, so links never break after a restart, and the frontend must be started with a flag that binds it to the network (not just the machine itself) for other computers to reach it at all.

Why SMTP port 465, not the more common 587

During testing, this network was found to block outbound port 587 (the standard STARTTLS mail port) while allowing port 465 (implicit SSL) through cleanly — a common ISP/router anti-spam measure. The email service is configured for port 465 (`SMTP_SSL`) accordingly. If deploying on a different network, both ports are worth testing before assuming either one is blocked.

Every email send happens in the background

Confirming an appointment, completing a lab result, or recording a payment all return an immediate response to the user interface — the actual email attempt runs afterward, as a background task, and never blocks or delays the action itself. If the mail server is temporarily unreachable, the underlying action (the appointment is still confirmed, the payment is still recorded) always succeeds regardless; only the email notification is affected, and every attempt — successful or not — is logged to the backend console as a permanent fallback record.

2. Frontend Documentation (React)

This section documents every screen in the completed React application, describing its purpose, UI elements, and the primary user actions it supports.

Folder structure (client-side/medcore-hms/src)

- components/layout/ — NavBar.jsx, SideBar.jsx, Footer.jsx, ThemeToggle.jsx (shared shell, used on every authenticated page)
- components/shared/ — PatientPicker.jsx, StaffPicker.jsx (search-and-select inputs reused across modules)
- components/ui/ — Avatar, Badge, Button, Card, EmptyState, Input, Modal, Table, Tabs, ToastContainer (base design-system primitives)
- modules/<name>/ — one folder per module (dashboard, patients, appointments, billing, pharmacy, laboratory, staff, reports, settings, notifications, refunds), each with a <Name>View.jsx entry point and its own components/ folder
- hooks/ — useAuth, useFetch, usePagination, useDebounce, useToast, useUnreadNotificationsCount
- services/ — one file per API resource (authService, patientService, appointmentService, billingService, pharmacyService, laboratoryService, staffService, reportService, notificationService, settingsService), all built on a single shared apiClient.js
- index.css — theme tokens for both themes, see Section 1.5
- App.jsx / main.jsx — application entry point, providers (Theme, Auth, Permissions), and routing

2.1 Global Layout Components

File location: src/components/layout/ — NavBar.jsx, SideBar.jsx, Footer.jsx, ThemeToggle.jsx. These four are shared across every authenticated page in the app.

NavBar.jsx (persists on every authenticated page)

- Left: current page title
- Center: global search bar — searches patients, appointments, invoices by name/ID
- Right: ThemeToggle.jsx — a two-way switch between Light and Dark, sets data-theme on <html>
- Right: Notification bell — shows a live unread-count badge (polled every 30 seconds), links through to the full Notifications page
- Right: Quick-add "+ New" button — opens a menu: New Patient / New Appointment / New Invoice
- Right: Profile avatar — links to My Profile, Settings, Logout

SideBar.jsx (left-hand navigation)

Built from a single NAV_ITEMS array (one entry per module: id, icon, path), so adding a module to the sidebar is a one-line change. The brand mark at the top shows the logged-in user's role directly underneath the MEDCORE wordmark (e.g. "Admin", "Nurse", "Lab Technician"), so the currently active account is always visible at a glance.

- dashboard, patients, appointments, billing, pharmacy, laboratory, staff, reports, settings — each with its own icon. An Admin also sees a Refunds item, since the Refunds page is an Admin-only audit view (Section 2.6a)
- Items are filtered per the logged-in user's configured module permissions before rendering
- Active route styling: bg-hms-accent text-hms-accent-on with a soft accent-colored shadow. Inactive items: text-hms-text-dim, hover:bg-hms-surface-2 hover:text-hms-text

Footer.jsx (persists on every page)

- Left: © MEDCORE Hospital Management System, current year
- Center: app version number (v1.0.0)
- Right: Support / Contact link

2.2 Authentication Pages

Login — route: /login

Purpose: authenticate staff before entering the app.

- Hospital logo / branding
- Email or username input
- Password input
- "Forgot password?" link
- Login button — calls POST /api/auth/login, stores the JWT in sessionStorage, redirects to Dashboard
- Inline error message area for invalid credentials

Forgot / Reset / Set Password

- Forgot Password (/forgot-password) — email input, "Send Reset Link" button (POST /api/auth/forgot-password)
- Reset Password (/reset-password/{token}) — new password + confirm password fields (POST /api/auth/reset-password)
- Set Password (/set-password/{token}) — shown the first time a newly created staff account is activated; a temporary password is never generated or emailed — this link is the only way in. The activation link is sent for real, from the hospital's own Gmail mailbox; every send attempt is also printed to the backend console as a permanent fallback record, whether or not the email itself succeeds

2.3 Dashboard

File: src/modules/dashboard/DashboardView.jsx. Purpose: give any staff member an at-a-glance view of hospital activity on login. Built from five components:

- StatMetricCard.jsx — one card per key number: Total Patients, Today's Appointments, Pending Bills, Low-Stock Drugs, Pending Lab Results
- LiveAppointments.jsx — today's appointment schedule widget (patient, doctor, time, status badge)
- RevenueChartCard.jsx — revenue chart with a weekly/monthly toggle, pulled from GET /api/reports/revenue
- RecentPatientTable.jsx — last few patient registrations, name / ID / demographics / time-since-registered
- OperationalQuickActions.jsx — quick action buttons: New Patient, New Appointment, New Invoice, New Lab Order

2.4 Patient Records

Patients List — route: /patients

- PatientSearchFilter.jsx — search bar (name, patient ID, phone) plus filter chips: gender, status
- PatientDirectoryTable.jsx — table columns: Patient (avatar + name + ID), Age, Gender, Last Visit, Status
- "Add New Patient" button, pagination

Add / Edit Patient — route: /patients/new, /patients/{id}/edit

- Personal info: full name, date of birth, gender, phone, email, address
- Emergency contact: name, relationship, phone
- Medical info: blood group, known allergies
- Insurance info: provider, policy number
- Save button — POST /api/patients or PUT /api/patients/{id}

Patient Profile — component: PatientProfileTabs.jsx, route: /patients/{id}

Tabbed view of a single patient's full record, opened from the directory table. Each tab lazily fetches its own data, only firing once that tab is actually selected:

- Overview tab — demographics, contact info, insurance
- Medical History tab — chronological visit notes
- Appointments tab — past and upcoming appointments for this patient
- Billing tab — invoices and payment status for this patient (visible to Admin and Receptionist only, matching the module-wide billing restriction)
- Lab Results tab — completed and pending lab orders
- Prescriptions tab — prescription history and dispense status

2.5 Appointments

Appointments — route: /appointments

File: src/modules/appointments/AppointmentsView.jsx. Toggles between a List view and a hand-built month Calendar view — react-big-calendar was evaluated during development and ultimately removed entirely in favor of a purpose-built component with no external calendar dependency.

- AppointmentToolbar.jsx — Calendar/List view switch, status filter, and the "Book Appointment" button
- AppointmentList.jsx — the appointment rows: patient, doctor, appointment type, time, status badge
- CalendarMonthView.jsx — month grid only (no week/day/agenda modes). Prev / Next / Today navigation. Fetches only the visible month's appointments via a dedicated from/to date-range query parameter on GET /api/appointments, rather than pulling a large flat batch. Days containing appointments show small colored status dots and are clickable, opening a modal that lists that day's appointments; clicking an appointment in the modal opens its detail page. Empty days are inert by design — this calendar is a viewing surface, not a quick-create shortcut
- Color-coded status badges: Pending, Confirmed, Completed, Cancelled, No-show — using the fixed --hms-warning / --hms-success / --hms-info / --hms-danger tokens so status meaning never shifts with the active theme

Book / Edit Appointment — route: /appointments/new, /appointments/{id}/edit

- Patient selector (searchable)
- Doctor selector
- Date & time picker
- Appointment type (consultation, follow-up, emergency) and reason for visit
- Notes field
- Save button — POST /api/appointments or PUT /api/appointments/{id}

Automatic billing: booking an appointment (POST /api/appointments) automatically generates a one-line invoice priced from the Admin-configured fee for that appointment's type (Consultation / Follow-up / Emergency — see Section 2.11). There is no payment gate attached — the doctor can see the patient regardless of payment status. This exists purely so every visit is reflected in Reports > Revenue, the same way a lab order's invoice already is.

Appointment Details — route: /appointments/{id}

Status is a strict one-way state machine, enforced on the backend as well as reflected in which buttons the interface shows:

- Pending → Confirmed (only the doctor assigned to this specific appointment) or Cancelled (Admin, Receptionist, or the assigned Doctor)
- Confirmed → Completed (only the assigned doctor) or No-show (any role)
- Completed, Cancelled, and No-show are all terminal — no further status change is possible from any of them

A doctor viewing an appointment they are not assigned to sees a plain status message instead of action buttons (e.g. "Awaiting confirmation from Dr. X"); attempting the transition directly against the API from an unauthorized account is independently rejected by the backend with HTTP 403 or HTTP 400 for an invalid transition.

- Patient info snapshot and doctor assigned
- Consultation notes field
- "Edit Appointment" shortcut

2.6 Billing

Access note: every endpoint in the billing module — including simply listing or viewing an invoice — is restricted to the Admin and Receptionist roles. A doctor, nurse, pharmacist, or lab technician has no access to billing data of any kind, matching how a real front-desk billing workflow is handled.

Invoices List — route: `/billing`

- Table: Invoice #, Patient, Date Issued, Amount, Status (Paid / Unpaid / Partial)
- Filter by status
- "Create Invoice" button

Create / Edit Invoice — route: `/billing/new`, `/billing/{id}/edit`

- Patient selector
- Line items: description, quantity, and unit price — editable rows with an "Add Line Item" control
- Discount and tax fields; the total is always recalculated server-side on save, never trusted from the frontend's own display math
- Save button — POST `/api/invoices` or PUT `/api/invoices/{id}`

Invoice Detail / Payment — route: `/billing/{id}`

- Full invoice breakdown (line items, subtotal, discount, tax, total)
- "Record Payment" form: amount, method (cash, card, insurance, bank transfer), reference number
- Payment history table for this invoice
- "Download Receipt" — generates and downloads a real PDF receipt via the backend (reportlab), openable and printable through the browser's normal print dialog
- "Refund" — available once an invoice has at least one payment recorded; refund a drug-linked line item (restores pharmacy stock automatically) or enter a plain amount for a non-pharmacy invoice. Sends the patient a refund-confirmation email and moves the invoice to a terminal Refunded state

An invoice's status is recalculated automatically every time a payment is recorded: Unpaid until any payment exists, Partial once total payments are less than the invoice total, and Paid the moment total payments meet or exceed it — there is no separate manual "mark as paid" action. Void (Admin only) and Refund (Admin, Receptionist) both move an invoice to a terminal state — cancelled or refunded respectively — and neither can be reversed or repeated once set.

2.6a Refunds

Access note: this page and every endpoint behind it is restricted to Admin, matching the backend's own restriction (Section 3.3). It is intentionally left out of the configurable Roles & Permissions matrix (Section 1.4), so it can never be granted to any other role.

Refunds — route: `/refunds`

- Table: Invoice #, Patient, Source (Appointment / Lab Order / Prescription / Manual Invoice, inferred from the invoice's line items), Amount, Processed By, Refunded At
- Weekly / Monthly range toggle, matching the `?range` query parameter on GET `/api/refunds`
- Export to Excel button — calls GET `/api/refunds/export`

- Read-only: a refund is created from the Billing module's Invoice Detail page (Section 2.6), not from here — this page is purely an audit trail of refunds already processed

2.7 Pharmacy

Drug Inventory — route: /pharmacy

- Table: Drug Name, Category, Stock Qty, Unit Price, Expiry Date, Status
- Low-stock rows highlighted
- "Low Stock Only" filter toggle
- "Add Drug" button — visible only to the Pharmacist role. Every other role, including Admin, can view the full inventory but cannot add a new drug

Add / Edit Drug — route: /pharmacy/new, /pharmacy/{id}/edit

- Name, category, manufacturer
- Unit price, stock quantity, reorder threshold
- Expiry date
- Save button — POST /api/drugs (Pharmacist only) or PUT /api/drugs/{id} (Admin + Pharmacist)

Prescriptions — route: /pharmacy/prescriptions

- Table: Patient, Doctor, Date, Items, Status (Pending / Dispensed)
- "New Prescription" modal — patient + drug/quantity rows, doctor is always the logged-in account, never trusted from the request body
- "Dispense" action per pending row

Dispense — route: /pharmacy/prescriptions/{id}/dispense

- Prescription details and requested items
- Stock availability check per item before confirming — all items are validated before any stock is deducted, avoiding a partial dispense
- Confirm Dispense button — PATCH /api/prescriptions/{id}/dispense, deducts stock and logs the transaction. If a dispense pushes any drug's stock at or below its reorder threshold, a real low-stock notification is generated automatically

2.8 Laboratory

Access note: adding or editing a lab test in the catalog is restricted to the Lab Technician role only — not Admin, and not any other role — to keep pricing and naming consistent and avoid duplicate catalog entries.

Test Catalog — route: /laboratory/tests

- List of available lab tests: Name, Category, Price, Turnaround Time
- "Add Test" button — Lab Technician only
- Inline "Edit" action per row — Lab Technician only, allows updating a test's name, category, price, or turnaround time at any time

Lab Orders — route: /laboratory/orders

Any authenticated role may create a lab order (a nurse or receptionist frequently places the order on a doctor's behalf, recording which doctor is actually ordering it). Creating an order automatically generates a linked one-line invoice, priced from the test catalog.

- Table: Patient, Test, Ordered By, Date, Status (Pending / In Progress / Completed)
- "New Lab Order" button — patient, test, and ordering doctor selectors

- Pending orders show either a "Confirm" button (if the linked invoice is fully paid) or an "Awaiting Payment" label (if it is not) — the lab technician sees this directly on the order, with no need for billing access of their own

Payment gate: a lab technician cannot move an order from Pending into In Progress until the linked invoice has been paid in full. This is enforced on the backend (a premature confirm attempt returns HTTP 400), not just hidden in the interface — and it means every completed lab test also has a corresponding real invoice feeding into Reports' revenue figures.

Lab Result Entry — route: `/laboratory/orders/{id}/result`

- Order details (patient, test, ordering doctor)
- Result Data and Notes fields
- "Mark Completed & Notify Doctor" button — updates the order status and fires a real notification to the specific ordering doctor plus every Admin account

2.9 Staff Management

- Table: Name, Role, Department, Contact, Status (Active/Inactive) — route: `/staff`
- "Add Staff" button (Admin only)

Add / Edit Staff — route: `/staff/new`, `/staff/{id}/edit`

- Name, role (Admin / Doctor / Nurse / Receptionist / Pharmacist / Lab Technician), department, phone, email
- No temporary password is ever generated or stored — a brand-new staff account is created in an unusable state until the staff member follows their emailed activation link and sets their own password (every send is also logged to the backend console as a fallback, in case the mailbox is unreachable)

2.10 Reports — route: `/reports`

- Report type selector: Revenue, Patient Registrations, Appointments, Pharmacy Stock, Lab Utilization
- Weekly / Monthly range toggle
- Chart + underlying summary cards, whose labels and formatting are driven by a per-report-type lookup rather than a fixed set of hardcoded cards
- Export to PDF / Excel button

Revenue figures: the Revenue report totals every invoice issued within the selected period, regardless of whether it has actually been paid yet. An unpaid invoice — including one auto-generated by a lab order awaiting payment — is already counted as revenue the moment it is created, not the moment it is collected. The one exception is a refunded invoice: the moment a refund is processed (Section 2.6, Billing), that invoice is excluded from Reports > Revenue from then on, regardless of how much of it had previously been counted.

2.11 Settings — route: `/settings`

- Hospital Profile tab — name, address, contact details (logo upload is a placeholder, not yet supported)
- User Preferences tab — notification preferences (theme is set from the always-visible Navbar toggle, not duplicated here)
- Roles & Permissions tab — a checkbox matrix of which modules each role can access; changes apply the next time a staff member logs in
- Appointment Fees tab (Admin only) — sets the fixed Consultation / Follow-up / Emergency fees auto-charged when an appointment is booked; not shown at all to any non-admin role, matching the backend's own restriction on both reading and writing this setting
- Backup & Data tab — download a full backup of hospital records

2.12 My Profile — route: `/profile`

- Edit own name, phone, department

- Change password form (current password, new password, confirm)

2.13 Notifications & Patient Emails — route: /notifications

Two separate channels exist for letting people know something happened — in-app notifications for staff, and email for patients. Neither is decorative; both are generated by specific backend actions.

In-app notifications (staff, shown via the Navbar bell)

- A drug's stock dropping to or below its reorder threshold — sent to every Admin and Pharmacist
- A new prescription being created — sent to every Admin and Pharmacist
- A new lab order being created — sent to every Admin and Lab Technician
- A lab result being marked completed — sent to the specific ordering doctor, plus every Admin
- Full list of notifications with type icon, message, and timestamp; "Mark as read" per item and "Mark all as read"
- The Navbar bell shows a live unread-count badge, refreshed automatically every 30 seconds

Patient emails (sent from the hospital's own Gmail mailbox — see Section 1.6)

- An appointment being confirmed by its assigned doctor — the patient is emailed the confirmed date, time, and doctor's name
- A lab result being marked completed — the patient is emailed a notice to come collect their results in person; no report file is attached or uploaded anywhere, by design
- An invoice reaching fully Paid — the patient is emailed their PDF receipt as an attachment. This applies to every invoice type identically (a regular consultation invoice, an appointment's auto-generated fee, or a lab order's auto-generated invoice), since all three are just rows in the same invoices table
- If a patient has no email on file, the corresponding email is silently skipped — it never blocks or fails the underlying action (confirming the appointment, completing the result, recording the payment)

Not yet built: time-based appointment reminders ("starts in X minutes") are not implemented. Doing so correctly requires background scheduler infrastructure — for example a periodic job checking for upcoming appointments — which does not currently exist in the project and was deliberately left for future work rather than built as a fixed-time hack.

2.14 Error Page — route: * (404)

- Friendly "page not found" message with a link back to Dashboard

3. Backend Documentation (FastAPI + Uvicorn)

The backend is a FastAPI application served by Uvicorn, exposing a REST API under `/api`. Every endpoint below returns JSON and expects a valid JWT in the Authorization header unless it is an auth endpoint itself.

3.1 Architecture Overview

- `main.py` — creates the FastAPI app, mounts routers, configures CORS and middleware
- `routers/` — one router module per domain (auth, users, staff, patients, appointments, billing, pharmacy, laboratory, reports, dashboard, notifications, settings)
- `models/` — SQLAlchemy async models mapping to database tables
- `schemas/` — Pydantic request/response models for validation and OpenAPI docs
- `services/` — business logic kept out of routers: `stock_service.py` (dispense validation and stock deduction), `invoice_service.py` (server-side total calculation, invoice numbering), `notification_service.py` (creating notifications for a role group or a specific user plus admins)
- `dependencies.py` — shared dependencies: `get_current_active_user`, `require_role(...)` guards, DB session
- `core/security.py` — Argon2 password hashing and JWT create/verify (access, refresh, and separate short-lived "action" tokens for set-password / reset-password links), all in one place so nothing else in the app calls the hashing or JWT libraries directly
- FastAPI auto-generates interactive API docs at `/docs` (Swagger) and `/redoc`

Deployment note: the system runs against a single PostgreSQL database, migrated with Alembic.

3.2 Database Schema

users (staff)

Field	Type	Notes
id	UUID, PK	
full_name	varchar	
email	varchar, unique	Used for login
password_hash	varchar, nullable	Argon2 hash. Nullable — an account has no usable password until the staff member sets their own via the emailed activation link
role	enum	admin, doctor, nurse, receptionist, pharmacist, lab_technician
department	varchar	Nullable
phone	varchar	
status	enum	active, inactive
is_verified	boolean	False until the account is activated
email_notifications / low_stock_alerts / appointment_reminders	boolean	Per-user preference flags
created_at / updated_at	timestamp	

patients

Field	Type	Notes
id	UUID, PK	
full_name / date_of_birth / gender	varchar / date / enum	
phone / email / address	varchar	
emergency_contact_name / relationship / phone	varchar	
blood_group / allergies	varchar / text	Nullable
insurance_provider / policy_no	varchar	Nullable
status	enum	active, inactive
created_at / updated_at	timestamp	

appointments

Field	Type	Notes
id	UUID, PK	
patient_id	FK → patients.id	
doctor_id	FK → users.id	
scheduled_at	timestamp	
type	enum	consultation, follow_up, emergency
reason	text	
status	enum	pending, confirmed, completed, cancelled, no_show — see the state machine in Section 2.5
notes	text	Nullable, consultation notes
created_at / updated_at	timestamp	

invoices, invoice_items & payments

Field	Type	Notes
invoices.id	UUID, PK	
invoices.patient_id	FK → patients.id	
invoices.status	enum	unpaid, partial, paid, cancelled, refunded — recomputed server-side after every payment; cancelled and refunded are terminal states set by the Void and Refund actions respectively, not by the normal payment-recompute path
invoices.discount / tax / total_amount	decimal	total_amount is always computed server-side, never trusted from the frontend
invoices.issued_at	timestamp	
invoice_items.invoice_id	FK → invoices.id	

Field	Type	Notes
invoice_items.description / qty / unit_price	varchar / int / decimal	Manually entered (e.g. a custom procedure), or auto-generated — "Consultation Fee" / "Follow Up Fee" / "Emergency Fee" from booking an appointment, or "Lab Test: <name>" from placing a lab order
invoice_items.drug_id	FK → drugs.id, nullable	Set only when this line item came from a pharmacy dispense; null for a manual or appointment/lab-generated line. Used by the refund workflow (Section 3.3) to know which items are stock-returnable
payments.invoice_id	FK → invoices.id	
payments.amount / method / reference / paid_at	decimal / enum / varchar / timestamp	method: cash, card, insurance, transfer

refunds

Field	Type	Notes
id	UUID, PK	
invoice_id	FK → invoices.id	The invoice this refund was issued against
amount	decimal	Always computed server-side. If items were returned, this is the sum of returned quantity × unit price; if not, it is the amount entered directly, capped at the amount actually paid
reason	varchar, nullable	Free text. For a stock-return refund, auto-prefixed with which items and quantities were returned, before any reason text the staff member typed
processed_by	FK → users.id	The Admin or Receptionist account that processed the refund
refunded_at	timestamp	
created_at / updated_at	timestamp	

One refund per invoice; refunding is terminal — a refunded invoice cannot be refunded again. If any refunded line item was a pharmacy drug (invoice_items.drug_id is set), the returned quantity is added back to that drug's stock_qty automatically. A refund also flips the parent invoice's status to refunded and excludes it from Reports > Revenue from that point on (see Sections 2.10 and 4).

drugs & prescriptions

Field	Type	Notes
drugs.id	UUID, PK	
drugs.name / category / manufacturer	varchar	
drugs.unit_price / stock_qty / reorder_threshold	decimal / int / int	A low-stock notification fires the moment stock_qty transitions to at or below reorder_threshold
drugs.expiry_date	date	
prescriptions.id / patient_id / doctor_id	UUID / FK / FK	
prescriptions.status	enum	pending, dispensed
prescription_items.prescription_id / drug_id / qty	FK / FK / int	

lab_tests, lab_orders & lab_results

Field	Type	Notes
lab_tests.id	UUID, PK	
lab_tests.name / category / price / turnaround_time	varchar / varchar / decimal / varchar	
lab_orders.id	UUID, PK	
lab_orders.patient_id / test_id / ordered_by	FK / FK / FK	ordered_by → users.id, must be a doctor
lab_orders.status	enum	pending, in_progress, completed
lab_orders.invoice_id	FK → invoices.id, nullable	Links each order to its auto-generated invoice; the payment gate on confirming an order checks this invoice's status. Nullable so orders created before this feature existed are unaffected
lab_results.order_id	FK → lab_orders.id, unique	
lab_results.result_data / notes / report_file_url	text / text / varchar	result_data is free-text (originally JSON in an earlier draft, corrected to match how the frontend has always submitted it — a plain string, not structured data). report_file_url is reserved for a future file-upload feature that was ultimately never built; patients collect physical results in person instead
lab_results.completed_at	timestamp	

notifications

Field	Type	Notes
id	UUID, PK	
user_id	FK → users.id	Recipient — one row per recipient, even when the same event notifies several people
type	enum	low_stock, lab_result, appointment_reminder, general
message_key / message_params	varchar / json	A template key plus its variable data, resolved into a final sentence on the frontend at render time
is_read	boolean	Default false
created_at	timestamp	

app_settings

A single generic key/value table backs four otherwise-unrelated settings screens, rather than one dedicated table per screen — a deliberate space-saving choice given how small and simple each setting is.

Field	Type	Notes
id	UUID, PK	
key	varchar, unique	"hospital_profile", "role_permissions", or "appointment_fees" — one row per key
value	json	The actual setting data, shaped differently per key (e.g. the fee amounts, or the role-to-module permission map)

3.3 API Endpoints

Auth — /api/auth

Method	Endpoint	Description	Auth Role
POST	/api/auth/login	Authenticate, returns access + refresh JWT	Public
POST	/api/auth/refresh	Exchange refresh token for a new access token	All staff
POST	/api/auth/logout	Invalidate the refresh token	All staff
GET	/api/auth/me	Return the current logged-in user's profile	All staff
POST	/api/auth/forgot-password	Send a password reset link/token by email	Public
POST	/api/auth/reset-password	Set a new password using a reset token	Public
POST	/api/auth/set-password	Set the first password on a newly created account	Public (token-verified)
POST	/api/auth/change-password	Change the current user's own password (requires the current password)	All staff

Users — /api/users

Method	Endpoint	Description	Auth Role
PUT	/api/users/me	Update the current user's own profile info	All staff

Staff — /api/staff

Method	Endpoint	Description	Auth Role
GET	/api/staff	List staff, filterable by role/department/status	All staff
POST	/api/staff	Create a new staff member (no password — activation link only)	Admin
GET	/api/staff/{id}	Get one staff member's details	All staff
PUT	/api/staff/{id}	Update a staff member's details	Admin
DELETE	/api/staff/{id}	Deactivate a staff member (soft delete — identical effect to PATCH /status with "inactive"; a staff member referenced by appointments, prescriptions, or lab orders cannot be safely hard-deleted without orphaning historical clinical records)	Admin
PATCH	/api/staff/{id}/status	Activate / deactivate a staff account	Admin

Patients — /api/patients

Method	Endpoint	Description	Auth Role
GET	/api/patients	List / search patients, paginated	All staff
POST	/api/patients	Register a new patient	Admin, Receptionist
GET	/api/patients/{id}	Get one patient's full record	All staff

Method	Endpoint	Description	Auth Role
PUT	/api/patients/{id}	Update patient details	Admin, Receptionist
DELETE	/api/patients/{id}	Deactivate a patient (soft delete — sets status to inactive, does not remove the record)	Admin
GET	/api/patients/{id}/history	Get medical history timeline	All staff
GET	/api/patients/{id}/appointments	Get this patient's appointments	All staff
GET	/api/patients/{id}/invoices	Get this patient's invoices	Admin, Receptionist
GET	/api/patients/{id}/lab-results	Get this patient's lab results	All staff
GET	/api/patients/{id}/prescriptions	Get this patient's prescriptions	All staff

Appointments — /api/appointments

Method	Endpoint	Description	Auth Role
GET	/api/appointments	List appointments — filterable by status, doctor_id, and a from/to calendar-date range (used by the month Calendar view)	All staff
POST	/api/appointments	Book a new appointment; also auto-generates a one-line invoice priced from the Admin-configured fee for that appointment type	Admin, Receptionist
GET	/api/appointments/{id}	Get one appointment's details	All staff
PUT	/api/appointments/{id}	Reschedule / edit an appointment. Admin and Receptionist may edit any appointment; a Doctor may only edit one they are personally assigned to. If the date/time actually changes, the patient is emailed	Admin, Receptionist, or the assigned Doctor
DELETE	/api/appointments/{id}	Legacy hard-cancel route, kept for API completeness. The frontend's Cancel button actually goes through PATCH /status below, which also voids the linked invoice and emails the patient	Admin, Receptionist, Doctor
PATCH	/api/appointments/{id}/status	Enforced state-machine transition. Confirm and Complete: only the specific doctor assigned to that appointment. No-show: any authenticated role. Cancel: Admin, Receptionist, or the assigned Doctor — so a patient cancelling at the front desk doesn't need the doctor available to action it. Both only from the correct prior status	See description
GET	/api/appointments/today	Today's appointment list, for the Dashboard widget	All staff

Billing — /api/invoices

Every endpoint in this router — including plain reads — requires Admin or Receptionist, with one exception: voiding an invoice (DELETE below) is Admin-only. No other role has any access to billing data.

Method	Endpoint	Description	Auth Role
GET	/api/invoices	List invoices, filterable by status	Admin, Receptionist
POST	/api/invoices	Create a new invoice with line items	Admin, Receptionist
GET	/api/invoices/{id}	Get one invoice's full breakdown	Admin, Receptionist
PUT	/api/invoices/{id}	Edit invoice line items	Admin, Receptionist
DELETE	/api/invoices/{id}	Void an invoice	Admin
POST	/api/invoices/{id}/payments	Record a payment against an invoice; recalculates status	Admin, Receptionist
GET	/api/invoices/{id}/payments	List payments made on an invoice	Admin, Receptionist
POST	/api/invoices/{id}/refund	Refund a paid invoice. Two modes: pass items (returned drug line items) to compute the amount automatically and restore stock, or pass amount directly for a non-pharmacy refund (appointment/lab/manual invoice). One refund per invoice; sets status to refunded	Admin, Receptionist
GET	/api/invoices/{id}/receipt	Generate and download a PDF receipt	Admin, Receptionist

Pharmacy — /api/drugs, /api/prescriptions

Method	Endpoint	Description	Auth Role
GET	/api/drugs	List drug inventory	All staff
POST	/api/drugs	Add a new drug to inventory	Pharmacist
GET	/api/drugs/{id}	Get one drug's details	All staff
PUT	/api/drugs/{id}	Update drug details / restock	Admin, Pharmacist
DELETE	/api/drugs/{id}	Remove a drug from inventory	Pharmacist
GET	/api/drugs/low-stock	List drugs at or below reorder threshold	All staff
GET	/api/prescriptions	List prescriptions, filterable by status	All staff
POST	/api/prescriptions	Create a prescription — fires a notification to Admin + Pharmacist	Doctor
GET	/api/prescriptions/{id}	Get one prescription's details	All staff
PATCH	/api/prescriptions/{id}/dispense	Dispense medication; deducts stock and fires a low-stock notification if any item newly crosses its threshold	Pharmacist

Laboratory — /api/lab-tests, /api/lab-orders

Method	Endpoint	Description	Auth Role
GET	/api/lab-tests	List the test catalog	All staff
POST	/api/lab-tests	Add a new test type to the catalog	Lab Technician
PUT	/api/lab-tests/{id}	Edit a test's name, category, price, or turnaround time	Lab Technician
DELETE	/api/lab-tests/{id}	Remove a test from the catalog	Lab Technician
GET	/api/lab-orders	List lab orders, filterable by status	All staff
POST	/api/lab-orders	Order a lab test for a patient; auto-generates a linked invoice priced from the catalog, and fires a notification to Admin + Lab Technician	All staff
GET	/api/lab-orders/{id}	Get one lab order's details, including its linked invoice's payment status	All staff
PUT	/api/lab-orders/{id}/result	Submit result data / notes for an order	Lab Technician, Admin
PATCH	/api/lab-orders/{id}/status	Update status. pending → in_progress is blocked (HTTP 400) unless the linked invoice is fully paid. completed fires a notification to the ordering doctor + all Admins	Lab Technician, Admin

Reports — /api/reports

Method	Endpoint	Description	Auth Role
GET	/api/reports/revenue	Revenue totals over a date range (counts invoices at the moment they're issued, not when paid)	Admin
GET	/api/reports/appointments	Appointment volume & status breakdown	Admin
GET	/api/reports/patients	New patient registrations over time	Admin
GET	/api/reports/pharmacy	Stock levels & dispensing trends	Admin
GET	/api/reports/lab	Lab test volume & completion stats	Admin
GET	/api/reports/export	Export any of the above as PDF or Excel	Admin

Dashboard — /api/dashboard

Method	Endpoint	Description	Auth Role
GET	/api/dashboard/stats	Stat-card numbers: patients, appointments, bills, stock, lab	All staff
GET	/api/dashboard/recent-patients	Last few registered patients	All staff
GET	/api/dashboard/today-appointments	Today's appointment schedule	All staff

Notifications — /api/notifications

Method	Endpoint	Description	Auth Role
GET	/api/notifications	List notifications for the current user	All staff

Method	Endpoint	Description	Auth Role
PATCH	/api/notifications/{id}/read	Mark one notification as read	All staff
PATCH	/api/notifications/read-all	Mark all notifications as read	All staff

Refunds — /api/refunds

Admin-only audit view of every refund already processed via POST /api/invoices/{id}/refund above. This router only lists and exports what has already happened — it is not where a refund is created, and it is intentionally excluded from the configurable Roles & Permissions matrix (Section 1.4), so it can never be granted to any role other than Admin.

Method	Endpoint	Description	Auth Role
GET	/api/refunds	List refunds within a date range (? range=weekly or monthly, default weekly). Each entry shows invoice #, patient, amount, reason, who processed it, and an inferred source (Appointment / Lab Order / Prescription / Manual Invoice)	Admin
GET	/api/refunds/export	Export the full refund history as an Excel (.xlsx) file	Admin

Settings — /api/settings

Method	Endpoint	Description	Auth Role
GET	/api/settings/hospital-profile	Get hospital name, address, contact details	All staff
PUT	/api/settings/hospital-profile	Update hospital profile	Admin
GET	/api/settings/preferences	Get the current user's own notification preferences	All staff
PUT	/api/settings/preferences	Update the current user's own preferences	All staff
GET	/api/settings/role-permissions	Get the module-permissions map — read access is open to every role, since every user's own sidebar is built from this on login	All staff
PUT	/api/settings/role-permissions	Change which modules each role can access	Admin
GET	/api/settings/appointment-fees	Get the fixed Consultation / Follow-up / Emergency fees	Admin
PUT	/api/settings/appointment-fees	Change the fixed fees used to auto-invoice new appointments	Admin
GET	/api/settings/backup	Download a JSON export (zipped) of core hospital tables — not a full database-grade backup	Admin

4. Known Limitations

Scope of This Documentation

This document is the complete and final description of MEDCORE HMS as delivered. Every module, screen, workflow, role permission, database table, and API endpoint the system actually has is described somewhere in the sections above — Sections 1 through 3 together form the full inventory of what this application does, and Section 4 above lists, in full, every acknowledged gap between what was built and what a more mature product might eventually include.

No feature, screen, workflow, integration, or capability should be assumed to exist in MEDCORE HMS unless it is explicitly described in this documentation. If a feature is not written above, it is not part of this build — not partially implemented, not planned for a specific date, and not available on request through configuration. Any expectation of functionality beyond what is documented here should be treated as incorrect, regardless of its source.

This is a deliberate choice, not an oversight. MEDCORE HMS was scoped, built, and tested against a defined set of requirements, and this documentation exists to describe that finished product accurately — not to advertise a roadmap, promise upcoming releases, or speculate about what the system could one day become. Whatever improvements or extensions may or may not follow this submission are a separate matter, outside the scope of what is being documented and evaluated here.

In short: this documentation is a description of a finished, working system as it stands today, not a preview of a system in progress. What is written above is what the application does. Nothing more should be inferred, and nothing less should be expected.

Developed by Mgboji Chinonso Joshua (BOHANOS)